Informe Técnico: Avance 4 del Proyecto ABP

Navegación Bidireccional con Lista Doblemente Enlazada

Jose Maldonado; Dario Chillogallo; Domenica Narváez; Galo Benitez; Jahir Campoverde

Universidad Nacional de Loja
Carrera de Computación
Asignatura: Estructura de Datos — Grupo N.° 03

31 de octubre de 2026

1. Introducción

En el presente avance del proyecto ABP se implementó una lista doblemente enlazada para permitir la navegación bidireccional entre elementos. Esta estructura permite avanzar, retroceder y eliminar nodos mediante referencias al elemento anterior y siguiente. Además, se desarrollaron pruebas unitarias para verificar su funcionamiento y se utilizó Git y GitHub para organizar el trabajo colaborativo, permitiendo que cada integrante contribuya al desarrollo del proyecto.

2. Repositorio y evidencia Git

Repositorio: https://github.com/Danilrox/APES_Estructura_de_Datos
Rama de trabajo: `feature/avance4-lista-doble-grupo03`
Último commit de la rama de trabajo: `c7512da`

Los integrantes desarrollaron sus aportes de manera colaborativa en la rama de trabajo indicada anteriormente. Los cambios se integraron mediante commits y se verificaron con las pruebas unitarias del proyecto.

3. Descripción de la estructura de datos

La `ListaDobleNavegacion` es una estructura de datos dinámica basada en enlaces bidireccionales. Cada elemento mantiene referencias hacia adelante y hacia atrás, lo que permite navegar eficientemente en ambas direcciones.

Componentes fundamentales

- **Nodo (`NodoDoble`):** almacena el dato y las referencias `siguiente` y `anterior`.
- **Cabeza (`cabeza`):** referencia al primer nodo. Si es `null`, la lista está vacía.
- **Cola (`cola`):** referencia al último nodo y permite insertar elementos al final en tiempo constante.
- **Cursor (`actual`):** señala el nodo seleccionado durante la navegación.

Métodos iniciales

- `constructor()`: inicializa cabeza, cola y actual en `null`, con longitud cero.
- `insertar(dato)`: crea un nuevo nodo y lo enlaza al final. Si la lista está vacía, establece cabeza, cola y actual en el nuevo nodo.
- `obtenerActual()`: devuelve el dato del nodo actual o `null` cuando no existe.

4. Análisis de complejidad

4.1. Análisis de complejidad de `avanzar()` y `retroceder()`

Los métodos `avanzar()` y `retroceder()` mueven el cursor `actual` de la `ListaDobleNavegacion` un nodo hacia adelante o hacia atrás. Cada nodo (`NodoDoble`) guarda dos referencias: una al nodo `siguiente` y otra al nodo `anterior`. Gracias a esto, el cursor puede moverse en ambos sentidos sin recorrer la lista.

Complejidad temporal: $O(1)$

Cada método ejecuta siempre la misma cantidad de pasos, sin importar cuántos nodos (n) tenga la lista:

1. Una comprobación: que `actual` no sea `null` y que exista el nodo vecino (`actual.siguiente` o `actual.anterior`).
2. Una reasignación de referencia: `this.actual = this.actual.siguiente` (o `.anterior`).
3. Un retorno: el dato del nuevo nodo, o `null` si se llegó a un extremo de la lista.

No hay ciclos (`for`, `while`) ni recursión. El número de operaciones es constante, por lo que $T(n) = c$ y la complejidad temporal es $O(1)$ tanto en el mejor como en el peor caso.

Complejidad espacial: $O(1)$

Ninguno de los dos métodos crea nodos, arreglos ni copias de los datos. Solo cambian a qué nodo apunta la referencia `actual`, que ya existía. La memoria adicional que usan

es constante, así que la complejidad espacial auxiliar es $O(1)$. Como solo se mueve una referencia, el Garbage Collector no recibe trabajo extra por navegar.

Comparación con una lista simplemente enlazada

En una lista simple cada nodo solo conoce al `siguiente`. Avanzar sigue siendo $O(1)$, pero para retroceder habría que recorrer la lista desde la cabeza hasta el nodo anterior al cursor, lo que en el peor caso cuesta $O(n)$. El enlace `anterior` de la lista doble evita ese recorrido, a cambio de una referencia extra por nodo.

Operación	Lista simple	Lista doble
`avanzar()`	$O(1)$	$O(1)$
`retroceder()`	$O(n)$	$O(1)$
Memoria por nodo	1 referencia	2 referencias

Tabla 1: Comparación de la navegación en lista simple y lista doble.

Verificación con pruebas unitarias

Las pruebas 2 y 3 de `tests/navegacion.test.js` comprueban este comportamiento (ver Tabla 2) con una lista de tres registros (101, 102 y 103). La prueba 2 avanza hasta la cola y verifica que en el límite superior se devuelva `null`. La prueba 3 avanza hasta la cola, retrocede hasta la cabeza y verifica que en el límite inferior también se devuelva `null`, sin que el cursor se pierda ni se lance un error.

4.2. Análisis de complejidad de eliminarActual()

El método `eliminarActual()` permite eliminar el nodo que se encuentra seleccionado mediante el cursor `actual` de la lista doblemente enlazada. Para realizar esta operación, utiliza las referencias `anterior` y `siguiente` del nodo seleccionado, permitiendo reconectar los nodos vecinos sin recorrer toda la estructura.

La operación contempla tres situaciones principales:

- **Eliminación de un nodo intermedio:** se conecta el nodo anterior con el siguiente.
- **Eliminación de la cabeza o la cola:** se actualiza el extremo correspondiente para conservar la estructura.
- **Eliminación del único nodo:** las referencias `cabeza`, `cola` y `actual` quedan en `null`.

Complejidad temporal: La operación tiene una complejidad de $O(1)$, debido a que accede directamente al nodo actual y modifica una cantidad constante de referencias. No necesita recorrer la lista.

Complejidad espacial: La complejidad auxiliar es $O(1)$, porque utiliza únicamente una referencia temporal al nodo eliminado, sin crear estructuras adicionales.

4.3. Gestión de memoria y Garbage Collector

Después de eliminar un nodo, el método establece sus referencias `siguiente` y `anterior` en `null`. Esta acción desvincula el nodo eliminado de la lista y evita que mantenga conexiones innecesarias con los demás elementos.

En JavaScript, el Garbage Collector (recolector de basura) recupera automáticamente la memoria de los objetos que ya no son accesibles desde el programa. Por lo tanto, al eliminar las referencias internas del nodo, se facilita su posterior recolección, siempre que no existan otras referencias externas que lo mantengan accesible.

Es importante señalar que establecer las referencias en `null` no libera inmediatamente la memoria; únicamente facilita que el recolector de basura identifique los objetos que ya no se necesitan.

De esta manera, el método `eliminarActual()` mantiene la integridad de la lista doblemente enlazada y contribuye a una gestión adecuada de la memoria.

5. Matriz de cobertura de pruebas

En la Tabla 2 se resumen las pruebas unitarias ejecutadas con `node --test` y en la Figura 1 se muestra la salida de la consola.

N.°	Prueba	Métodos evaluados	Estado
1	Inicialización	`insertar`, `obtenerActual`	[ok]
2	Avance secuencial	`avanzar`	[ok]
3	Retroceso secuencial	`avanzar`, `retroceder`	[ok]
4	Eliminación intermedia	`eliminarActual`, `retroceder`	[ok]
5	Casos límite (lista vacía)	Todos los métodos	[ok]

Tabla 2: Matriz de cobertura de las pruebas unitarias.

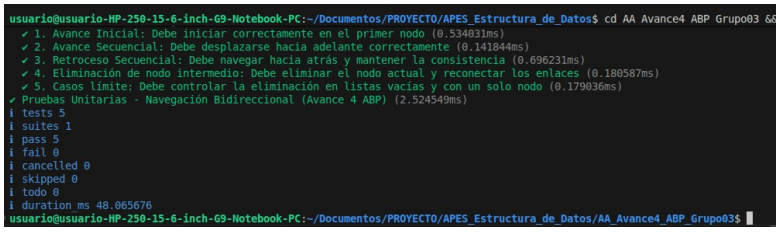


Figura 1: Resultado de `node --test` tests/navegacion.test.js: 5 de 5 pruebas aprobadas.

6. Conclusiones

- La lista doblemente enlazada permite navegar en ambas direcciones mediante referencias al nodo anterior y siguiente, facilitando el desplazamiento entre elementos.
- Las cinco pruebas unitarias ejecutadas permitieron verificar el correcto funcionamiento de la inicialización, el avance, el retroceso, la eliminación de nodos y los casos límite de la lista doblemente enlazada.
- El uso de Git y GitHub permitió organizar el trabajo colaborativo, integrar los aportes de los integrantes y mantener un registro de los cambios realizados.

7. Declaración de ética e inteligencia artificial

Para la organización y revisión del formato del presente informe se contó con el apoyo de ChatGPT (OpenAI), principalmente en aspectos relacionados con LaTeX. Cada integrante es responsable de su aporte y de verificar la información presentada.

Referencias

Bible, P. W. y Moser, L. (2023). *An Open Guide to Data Structures and Algorithms*. PALNI Open Press.

De la Cueva Hernández, V. M. y col. (2020). *Estructuras de datos y algoritmos fundamentales*. Tecnológico de Monterrey.

Hernández, M. y Baquero, L. E. (2022). *Estructuras de datos*. Ra-Ma Editorial.

Node.js. (2026). *Test Runner API & Memory Management*. https://nodejs.org/api/test.html